BLOCKCHAIN LABORATORY

Experiment No. 8

Configuring a Cryptocurrency Wallet using MetaMask and Integrating it with a DApp

Name: Nikhil Sunchu

Roll No.: 31

Class/section: D20A

Date:

1. Aim

Configuring a cryptocurrency wallet using MetaMask and integrating it with a decentralized application (DApp).

2. Theory

2.1. 1. Cryptocurrency Wallets

A cryptocurrency wallet manages blockchain accounts and allows users to approve transactions. MetaMask is a browser wallet used to connect to Ethereum-compatible networks, display the active account, and authorize contract interactions. The wallet does not store blockchain assets directly; it manages the credentials used to access an account.

2.2. 2. Sepolia Test Network

Sepolia is an Ethereum test network used to develop and test decentralized applications without using real Ether. Test ETH is used to pay transaction gas on the test network. In this experiment, MetaMask and Remix were connected to Sepolia Testnet, with chain ID `11155111`.

2.3. 3. Decentralized Applications and Smart Contracts

A decentralized application (DApp) provides an interface for interacting with blockchain smart contracts. Remix IDE can compile Solidity source code, deploy contracts, and call their functions. MetaMask acts as the wallet provider through which a user approves state-changing transactions.

2.4. 4. Contract Calls and Transactions

Reading a public state variable through its getter is a read-only operation and does not require a blockchain transaction. Calling a state-changing function, such as `updateName()` or `updateRoll()`, requires a transaction signed through the connected wallet and updates the contract state when mined.

3. Tools and Environment Used

- MetaMask browser extension.
- Sepolia Ethereum test network (chain ID 11155111).
- Remix IDE version 2.6.8.

- Solidity compiler version 0.8.34.
- Solidity source file: `StudentRecord.sol`.
- Contract deployed through Remix using the Browser Extension provider.

4. Tasks Performed

4.1. Task 1 – Configure the MetaMask Wallet

Opened MetaMask and selected the Sepolia test network. The wallet interface confirmed that Sepolia was selected. No real cryptocurrency was used in the experiment.

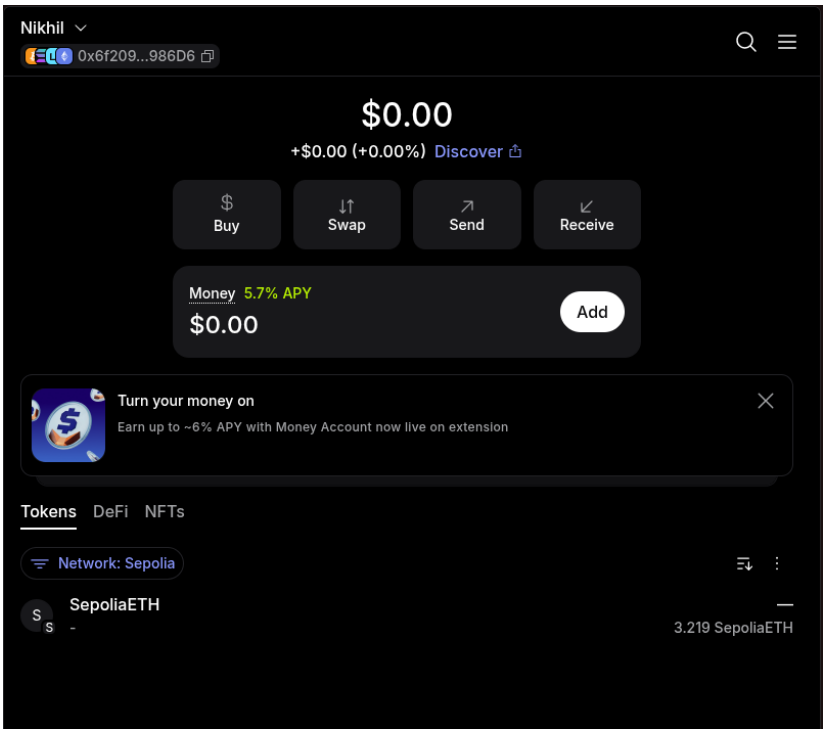


Figure 1: MetaMask wallet configured for the Sepolia test network

4.2. Task 2 – Open Remix IDE

Opened Remix IDE and used the File Explorer to create the Solidity source file for the DApp.

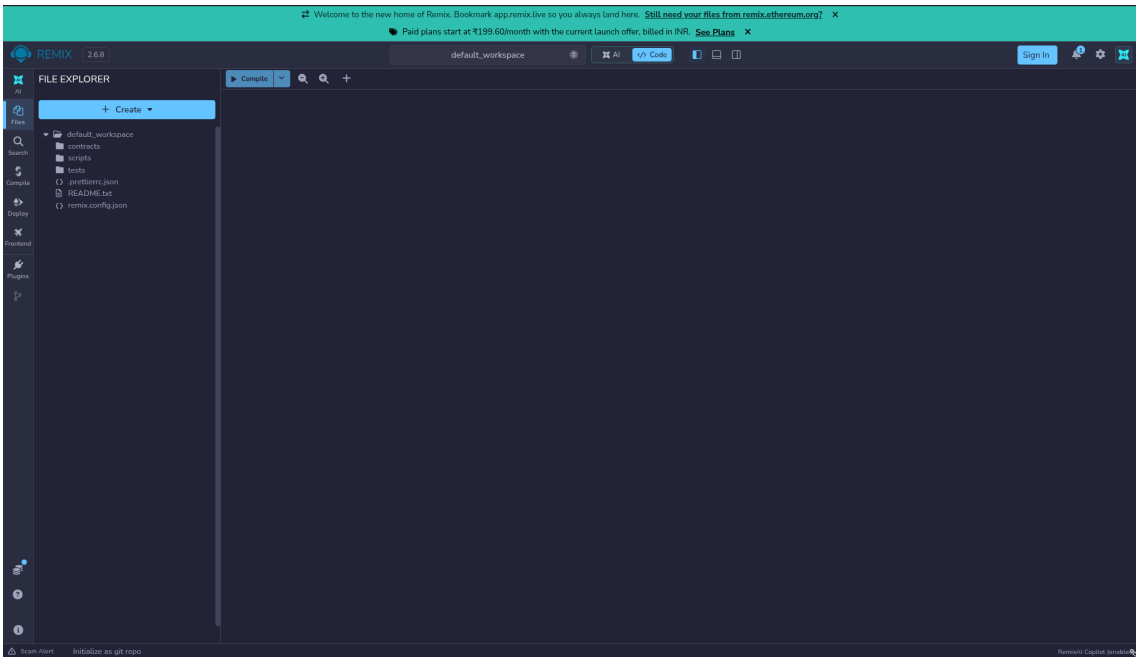


Figure 2: Remix IDE workspace

4.3. Task 3 – Write the Solidity Smart Contract

Created `StudentRecord.sol`. The contract stores a student's name and roll number, initializes both values through its constructor, and provides functions to update them.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract StudentRecord {
    string public studentName;
    uint256 public studentRoll;

    constructor(string memory _name, uint256 _roll) {
        studentName = _name;
        studentRoll = _roll;
    }

    function updateName(string memory _name) public {
        studentName = _name;
    }

    function updateRoll(uint256 _roll) public {
        studentRoll = _roll;
    }
}
```

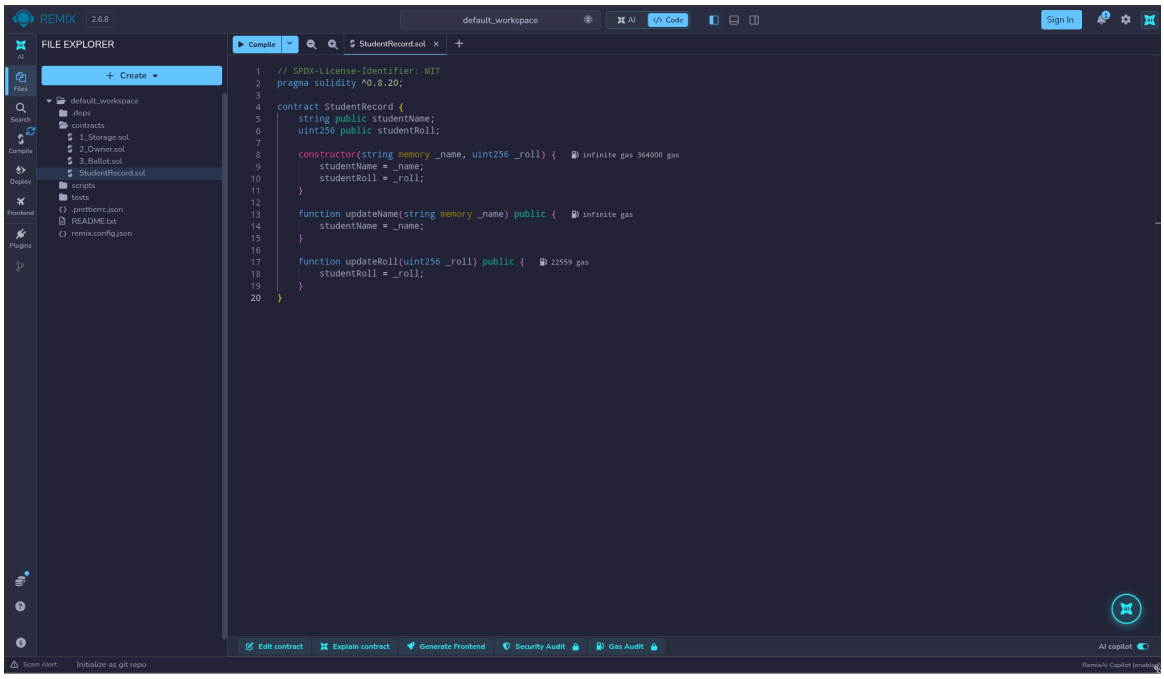


Figure 3: StudentRecord Solidity source code in Remix

4.4. Task 4 – Compile the Contract

Opened the Solidity Compiler panel and compiled `StudentRecord.sol` using compiler version 0.8.34. The green success indicator confirmed that compilation completed.

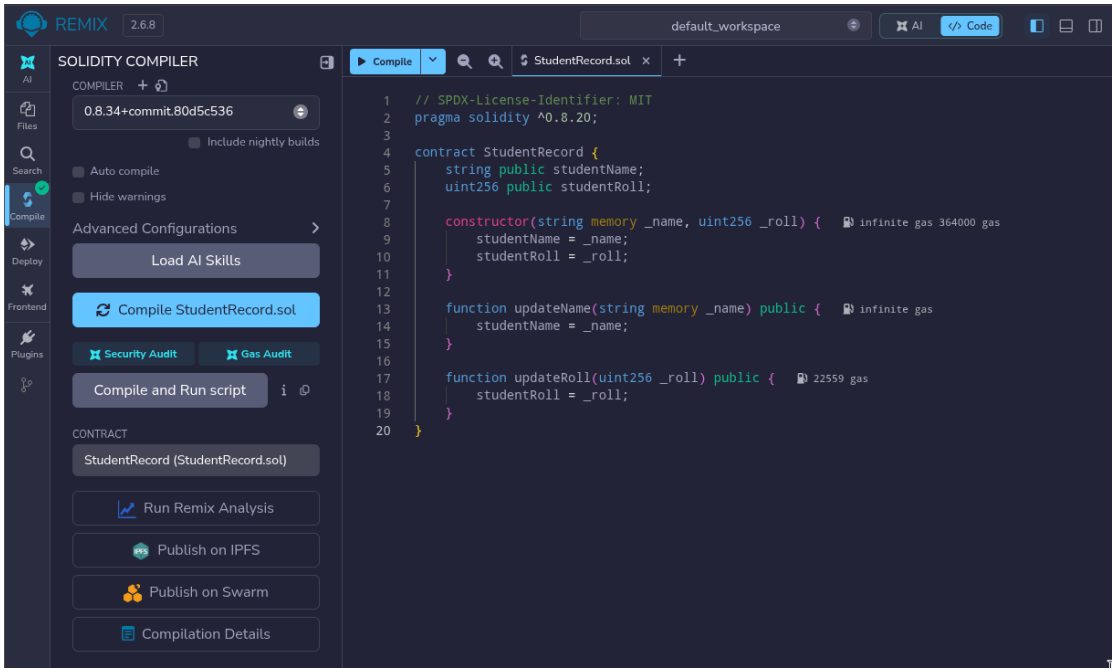


Figure 4: Successful compilation of StudentRecord

4.5. Task 5 – Connect Remix to MetaMask

In the Deploy & Run Transactions panel, selected the Browser Extension environment and the Sepolia Testnet – MetaMask provider. Remix displayed the connected account and Sepolia network.

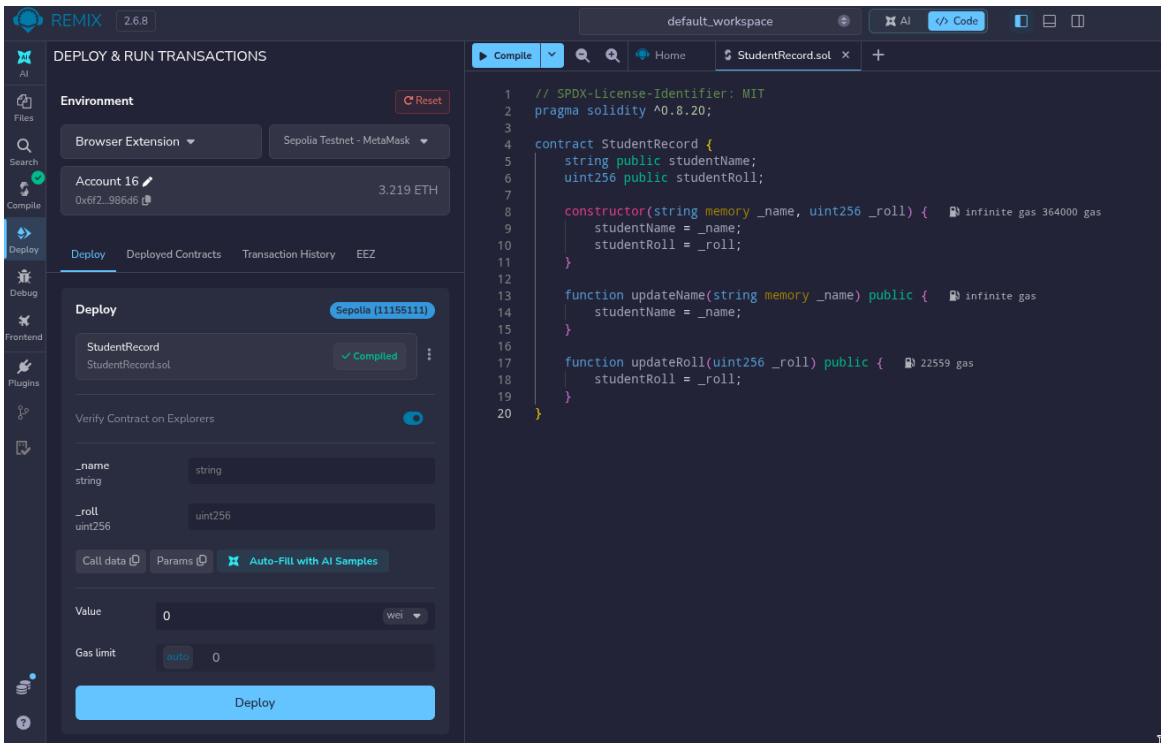


Figure 5: Remix connected to MetaMask on Sepolia

4.6. Task 6 – Deploy the Smart Contract

Entered `Nikhil Sunchu` and `31` as the constructor arguments and deployed the contract. Remix reported that the deployment transaction was mined and execution completed. The contract was available under Deployed Contracts on Sepolia.

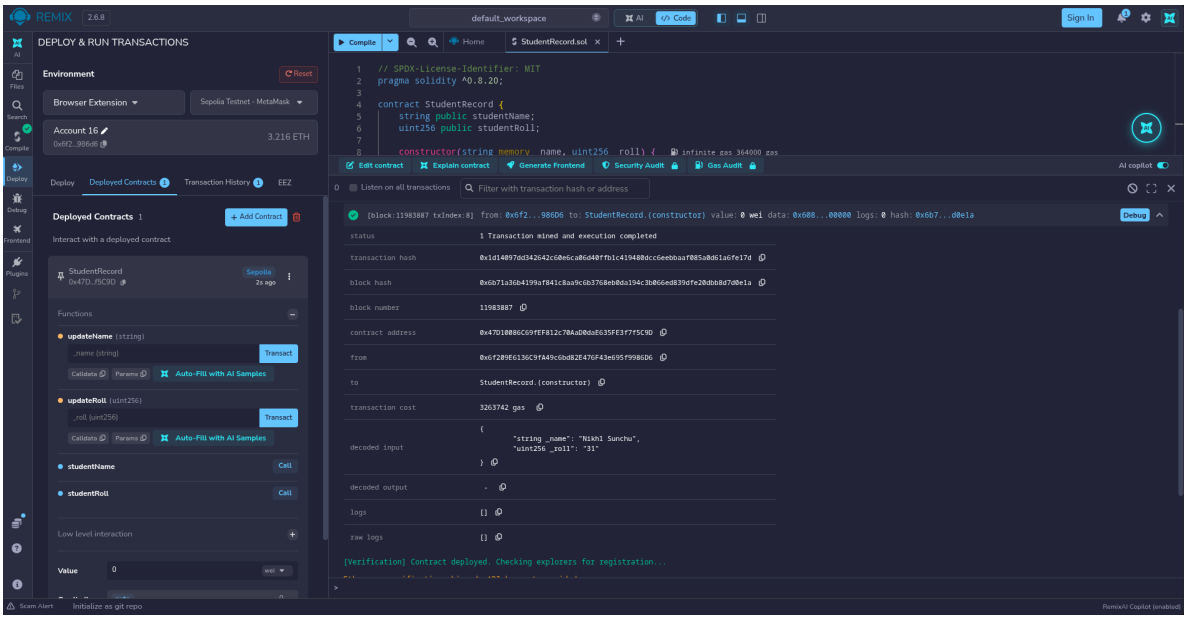


Figure 6: Successful deployment and transaction details

4.7. Task 7 – Verify the Initial Contract State

Called the public getter functions to read the initial values stored by the deployed contract. The returned values were `Nikhil Sunchu` and `31`.

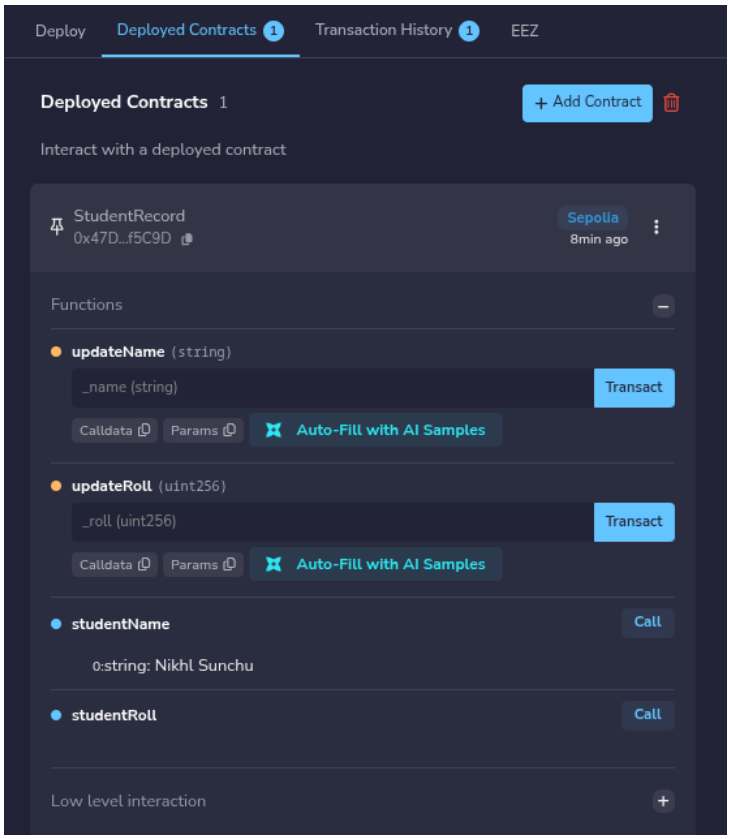


Figure 7: Initial student name returned by the contract

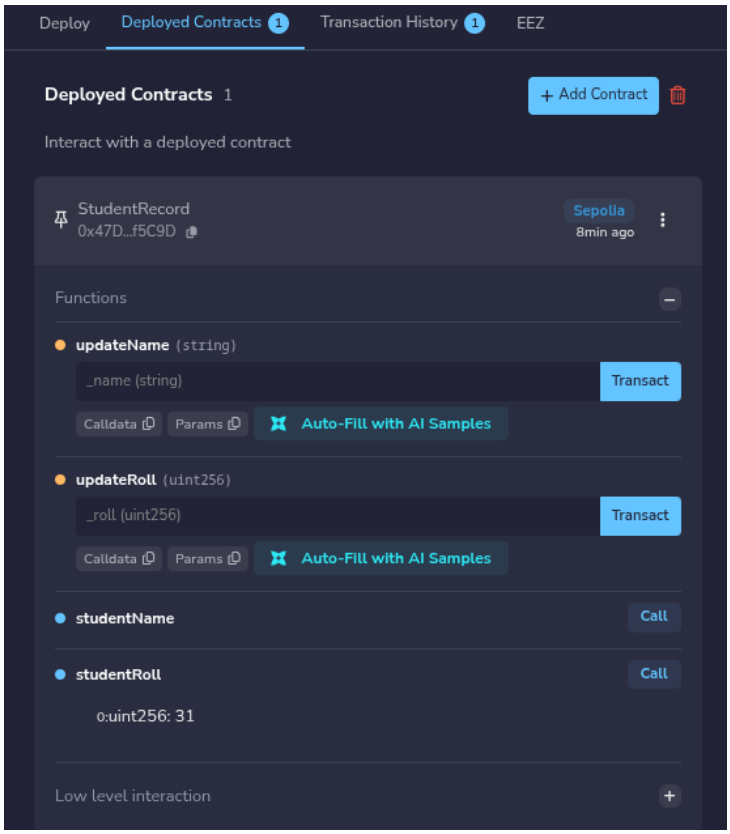


Figure 8: Initial student roll number returned by the contract

4.8. Task 8 – Execute State-Changing Transactions

Entered `Nikhil Sunchu IT` in `updateName()` and submitted the transaction through MetaMask. Then entered `32` in `updateRoll()` and submitted the second transaction. Remix showed that both transactions were mined and execution completed.

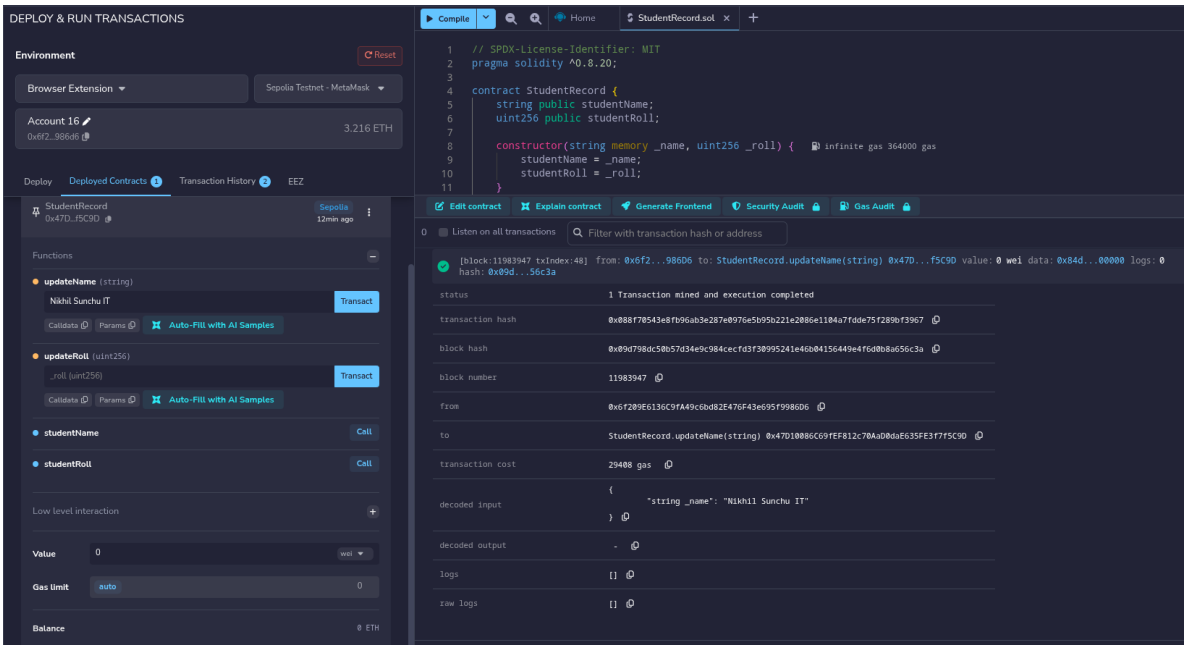


Figure 9: Successful updateName transaction

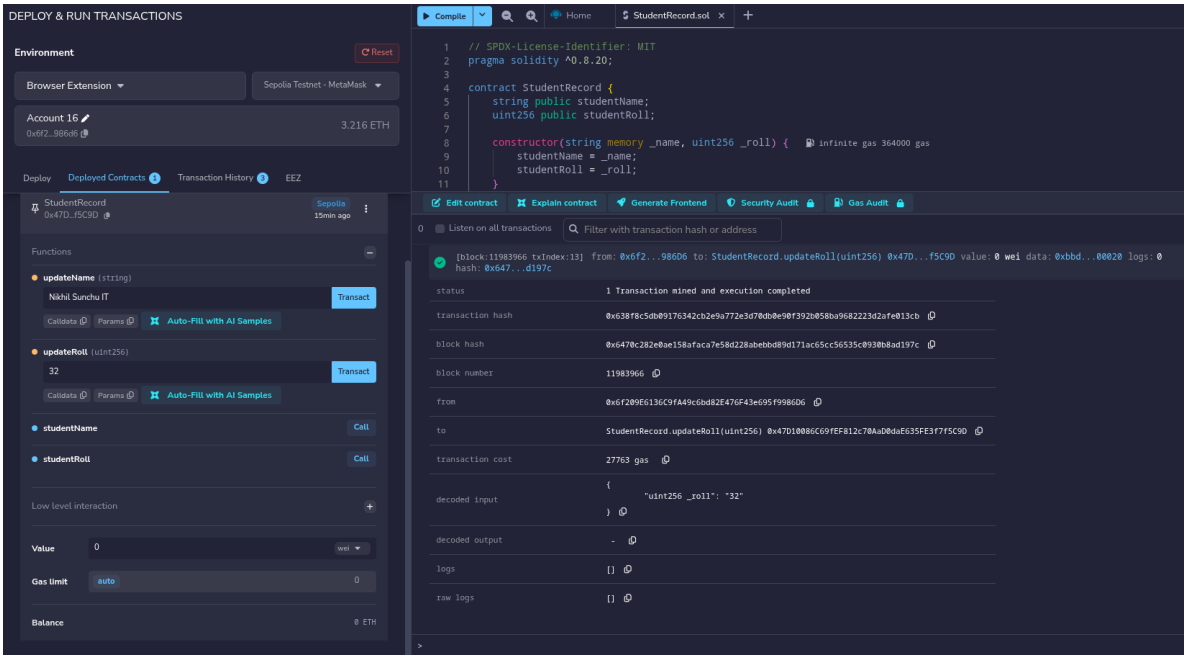


Figure 10: Successful updateRoll transaction

4.9. Task 9 – Verify the Updated Contract State

Called the getter functions after the transactions. The final stored student name was `Nikhil Sunchu IT`, and the final roll number was `32`.

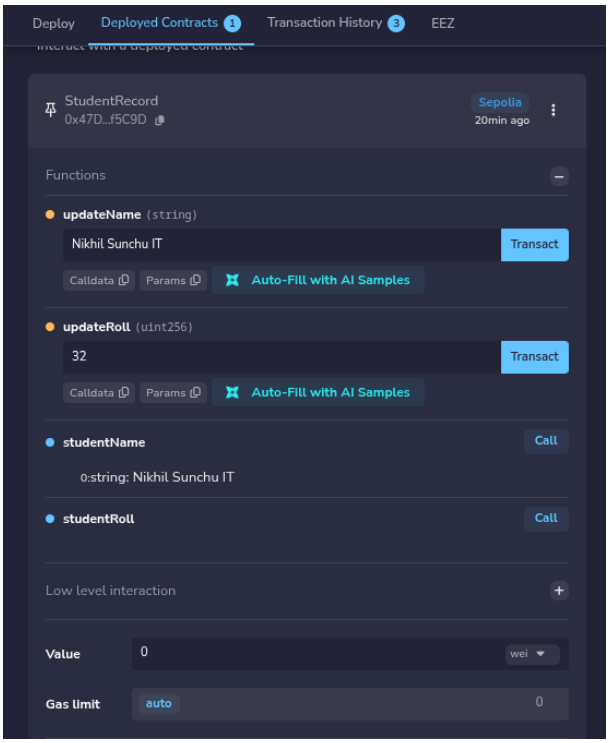


Figure 11: Final student name returned by the contract

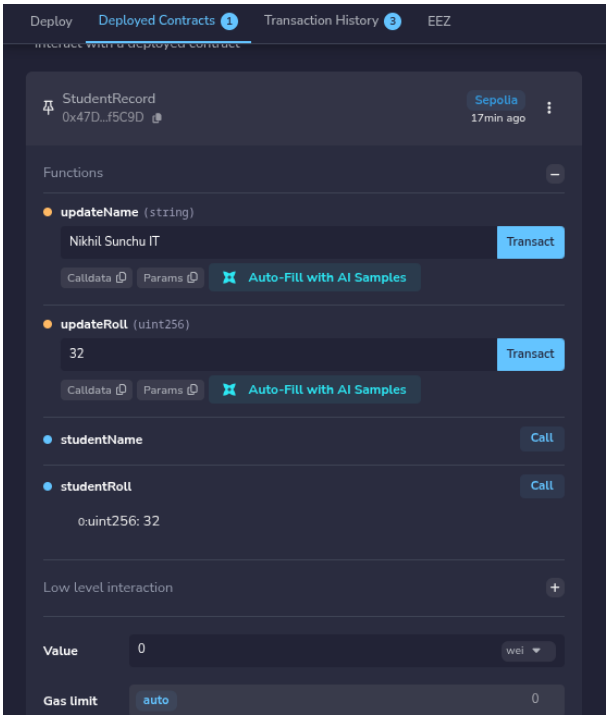


Figure 12: Final student roll number returned by the contract

5. Result

MetaMask was configured to use the Sepolia test network and successfully connected to Remix IDE. The `StudentRecord` smart contract was compiled and deployed to Sepolia with the initial values `Nikhil Sunchu` and `31`. State-changing transactions updated the name to `Nikhil Sunchu IT` and the roll number to `32`. The final values were verified through the contract's public getter functions.

6. Conclusion

The experiment provided practical experience in configuring a cryptocurrency wallet, connecting MetaMask to a DApp, and deploying and interacting with a Solidity smart contract on a test network. It demonstrated the distinction between read-only calls and state-changing transactions, and showed how MetaMask authorizes blockchain transactions.